



GHULAM ISHAQ KHAN INSTITUTE OF ENGINEERING SCIENCES AND
TECHNOLOGY

IC Design Verification

Pipelined RISC-V Processor Implementation

Five-Stage Pipeline with Hazard Detection, Forwarding, and Stall
Control

Submitted To: Eng Abbas Khan

Submitted By: Abdul Samad Abbasi
Student ID: 25ins-01030

Course: Computer Architecture – Module 2

Date: 28 June 2026

Abstract

This report presents a comprehensive design, implementation, verification, and analysis of a Five-Stage Pipelined RISC-V Processor implemented in Verilog HDL and simulated using Xilinx Vivado 2018.2. The processor implements a representative subset of the RV32I base integer instruction set including I-type (ADDI), R-type (ADD), S-type (SW), I-type load (LW), B-type (BEQ), and J-type (JAL) instructions. The pipeline executes all five stages – Fetch, Decode, Execute, Memory, and Writeback – concurrently, achieving a sustained throughput of approximately one instruction per clock cycle. A dedicated Hazard Unit handles all data hazards via forwarding and stalling, and all control hazards via pipeline flushing. This report covers the architecture of each pipeline stage in detail, the complete Verilog module hierarchy, hazard detection and resolution logic, simulation waveform analysis with TCL console trace, and a detailed register-file result table showing the final computed values for each instruction. All results are verified against Xilinx Vivado behavioural simulation.

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Design Philosophy	5
1.3	Key Features	5
2	Architecture and Datapath	6
2.1	Overall Pipeline Structure	6
2.2	Module Hierarchy	6
2.3	Pipeline Register Banks	7
3	Stage-by-Stage Implementation	8
3.1	Stage 1: Fetch	8
3.1.1	Purpose and Components	8
3.1.2	Signals Produced	8
3.1.3	Verilog Implementation	8
3.2	Stage 2: Decode	9
3.2.1	Purpose and Components	9
3.2.2	Control Unit Signals	9
3.2.3	Signals Produced (to ID/EX register)	9
3.2.4	Verilog Implementation (key excerpt)	9
3.3	Stage 3: Execute	10
3.3.1	Purpose and Components	10
3.3.2	Forwarding Mux Operation	10
3.3.3	PCSrc Generation	10
3.3.4	Branch-Target Calculation	11
3.3.5	Verilog Implementation	11
3.4	Stage 4: Memory	12
3.4.1	Purpose and Components	12
3.4.2	Verilog Implementation	12
3.5	Stage 5: Writeback	13
3.5.1	Purpose and Components	13
4	Pipeline Hazards and the Hazard Unit	14
4.1	Types of Pipeline Hazards	14
4.2	Hazard Unit Design	14
4.2.1	Forwarding Logic	15
4.3	Forwarding Condition Table	16
4.4	Load-Use Stall Mechanism	16
4.5	Control Hazard Resolution	17
5	Test Program and Instruction Memory	17

5.1	Instruction Memory Contents	17
5.2	Register File Final Values	18
6	Simulation Results and Waveform Analysis	18
6.1	TCL Console Trace Analysis	18
6.2	Zoomed Waveform: Early Pipeline (0–110 ns)	19
6.3	Zoomed Waveform: Branch and Jump Region (120–230 ns)	20
6.4	Overview Waveform	21
7	Performance Analysis	21
7.1	Cycle Count and CPI	21
7.2	Performance vs. Single-Cycle and Multi-Cycle	22
8	Summary and Conclusion	22
8.1	Verification Summary	22
8.2	Key Observations	23
8.3	Conclusion	23
A	Complete Verilog Source Code	23
A.1	Top-Level Pipeline Module	23
A.2	Register File	25
A.3	Pipeline Register Modules	25

List of Figures

1	Complete Vivado simulation waveform (0–220 ns) showing all pipeline signals.	6
2	Zoomed waveform (120–230 ns): BEQ branch flush and JAL jump flush events.	15
3	Zoomed waveform (0–110 ns): ADDI, ADD, SW, LW instructions and load-use stall.	20
4	Overview waveform showing all pipeline stages across the complete simulation.	21

List of Tables

1	Module Hierarchy and Functions	7
2	Pipeline Register Banks and Hazard Controls	7
3	Control Unit Output by Instruction Type	9
4	Execute-Stage Forwarding Mux Encodings	10
5	Hazard Unit Forwarding Condition Summary	16
6	Test Program Loaded in Instruction Memory	17
7	Final Register File Values After Program Execution	18

8	Key Events from TCL Console Trace	19
9	Performance Comparison Across Three Architectures	22
10	Design Verification Summary	22

1 Introduction

1.1 Project Overview

Pipelining is the single most important technique for improving processor performance. By subdividing instruction execution into discrete stages and processing multiple instructions simultaneously – one in each stage – a pipelined processor can approach a throughput of one instruction per clock cycle while each stage runs at a much shorter clock period than the equivalent single-cycle design. This project implements a **five-stage pipelined RISC-V processor** supporting the following RV32I instruction types:

- **I-Type ALU:** ADDI (add immediate)
- **R-Type:** ADD (register-to-register addition)
- **S-Type:** SW (store word)
- **I-Type Load:** LW (load word)
- **B-Type:** BEQ (branch if equal)
- **J-Type:** JAL (jump and link)

The design extends the single-cycle datapath by inserting pipeline registers between each stage and adding a Hazard Unit that resolves data hazards through forwarding and load-use stalling, and resolves control hazards by flushing the pipeline on taken branches and jumps.

1.2 Design Philosophy

In the pipelined design, five instructions execute concurrently – each in a different stage. The key insight is that functional units (instruction memory, register file, ALU, data memory) are each busy for only a fraction of the cycle in a sequential design; pipelining keeps them active simultaneously. The pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) latch all values produced in one stage so they are available for the next stage in the following cycle.

1.3 Key Features

- 32-bit RV32I architecture with 32 general-purpose registers (x0–x31)
- Separate Harvard instruction and data memories
- Five pipeline stages: Fetch (F), Decode (D), Execute (E), Memory (M), Writeback (W)
- Four inter-stage pipeline register banks with enable and flush controls
- Dedicated Hazard Unit implementing forwarding, load-use stall, and branch/jump flush
- 3:1 forwarding muxes on both ALU source operands (ForwardAE, ForwardBE)
- Complete Verilog RTL simulated in Xilinx Vivado 2018.2

2 Architecture and Datapath

2.1 Overall Pipeline Structure

The pipelined datapath is divided into five concurrent stages separated by pipeline register banks. At every rising clock edge, all registers latch simultaneously, so five different instructions advance through the pipeline in lock-step. Figure 1 shows the complete Vivado simulation waveform confirming all five stages operating concurrently.

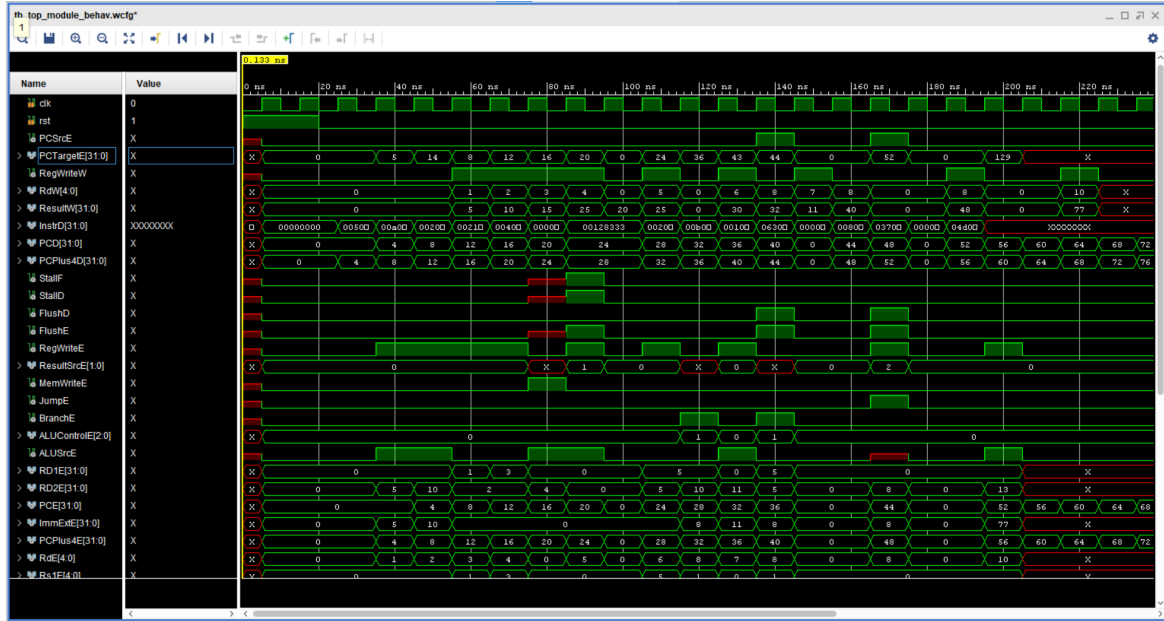


Figure 1: Complete Vivado simulation waveform (0–220 ns) showing all pipeline signals.

2.2 Module Hierarchy

Table 1 lists every Verilog module in the design together with its function and pipeline stage.

Table 1: Module Hierarchy and Functions

Module	Stage	Function	Type
top_pipe	All	Top-level integration, wire routing	Structural
fetch	F	PC, instruction memory, IF/ID regs	Mixed
decode	D	Register file, control unit, ID/EX regs	Mixed
execute	E	ALU, forwarding muxes, EX/MEM regs	Mixed
Memory	M	Data memory, MEM/WB regs	Mixed
WriteBack	W	Result mux, write-back to reg file	Combinational
hazard_unit	All	Forwarding, stall, flush logic	Combinational
pc_32bit	F	Program counter with stall enable	Sequential
INST_MEM	F	Instruction memory (ROM)	Combinational
data_mem	M	Data memory (RAM)	Sequential
REG_FILE	D	32×32-bit register file	Sequential
control_unit	D	Main decoder + ALU decoder	Combinational
Imm_Gen	D	Sign-extension (I/S/B/J formats)	Combinational
ALU	E	32-bit arithmetic logic unit	Combinational
adder	F/E	PC+4 and branch-target adder	Combinational
mux_2x1	F/E	2:1 multiplexer	Combinational
mux_3x1	E/W	3:1 multiplexer (forwarding/WB)	Combinational
Buff_RegF2	F	IF/ID register (with stall and flush)	Sequential
Buff_RegD	D	ID/EX register (with flush)	Sequential
Buff_Reg	E/M	EX/MEM and MEM/WB registers	Sequential

2.3 Pipeline Register Banks

Four sets of pipeline registers separate the five stages. Table 2 summarises the key registers and their hazard-control inputs.

Table 2: Pipeline Register Banks and Hazard Controls

Bank	Contents Latched	Stall Enable	Flush (CLR)
IF/ID (Buff_RegF2)	InstrD, PCD, PCPlus4D	StallID (hold)	FlushD (zero)
ID/EX (Buff_RegD)	Control signals, RD1E, RD2E, PCE, ImmExtE, PCPlus4E, RdE, Rs1E, Rs2E	–	FlushE (zero)
EX/MEM (Buff_Reg)	ALUResultM, WriteDataM, RdM, RegWriteM, ResultSrcM, MemWriteM, PCPlus4M	–	–
MEM/WB (Buff_Reg)	ALUResultW, ReadDataW, RdW, RegWriteW, ResultSrcW, PCPlus4W	–	–

3 Stage-by-Stage Implementation

3.1 Stage 1: Fetch

3.1.1 Purpose and Components

The Fetch stage retrieves the next instruction from instruction memory and updates the program counter. It integrates five components: a 2:1 PC mux (selects between PC+4 and the branch/jump target PCTargetE), a 32-bit program counter with stall enable, instruction memory, a PC+4 adder, and the IF/ID pipeline register bank (Buff_RegF2).

The PC mux input PCSrcE comes from the Execute stage and selects PCTargetE when a branch is taken or a JAL executes. The IF/ID registers have both a stall enable (StallD, holds the registers frozen during a load-use stall) and a clear input (FlushD, zeroes the registers to insert a bubble when a branch or jump flushes the pipeline).

3.1.2 Signals Produced

- **InstrD** – fetched instruction word, propagated to Decode
- **PCD** – current PC value, propagated to Decode (needed for branch target in RISC-V)
- **PCPlus4D** – PC+4 return address, propagated to Decode

3.1.3 Verilog Implementation

```

1 module fetch(
2     input clk, PCSrcE, StallF, StallD, FlushD, rst,
3     input [31:0] PCTargetE,
4     output [31:0] InstrD, PCD, PCPlus4D
5 );
6     wire [31:0] PCPlus4F, PC_F, PCF, RD_F;
7
8     mux_2x1 m1 (.a(PCPlus4F), .b(PCTargetE), .y(PC_F), .s(
9         PCSrcE));
10    pc_32bit p1 (.clk(clk), .rst(rst), .en(StallF),
11        .prev_value(PC_F), .count(PCF));
12    INST_MEM IM1 (.PC(PCF), .Instruction_Code(RD_F));
13    adder a1 (.A(PCF), .B(32'd4), .sum(PCPlus4F));
14
15    Buff_RegF2 Reg_InstrF (.clk(clk), .rst(rst), .CLR(FlushD),
16        .en(StallD), .X(RD_F), .Y(InstrD));
17    Buff_RegF2 Reg_PCF (.clk(clk), .rst(rst), .CLR(FlushD),
18        .en(StallD), .X(PCF), .Y(PCD));
19    Buff_RegF2 Reg_PCPlus4F (.clk(clk), .rst(rst), .CLR(FlushD),
20        .en(StallD), .X(PCPlus4F), .Y(PCPlus4D))
21        ;
22 endmodule
    
```

Listing 1: fetch.v – Fetch Stage Module

3.2 Stage 2: Decode

3.2.1 Purpose and Components

The Decode stage extracts source and destination register addresses, reads the register file, decodes the opcode into control signals, and sign-extends the immediate field. It contains: the Control Unit, Register File, Immediate Generator, and the ID/EX pipeline register bank (Buff.RegD). The register file write port is also located here, receiving RegWriteW, RdW, and ResultW from the Writeback stage on the falling edge of the clock to avoid a read-after-write conflict within the same cycle.

3.2.2 Control Unit Signals

The Control Unit combinationally decodes InstrD[6:0] (opcode) and InstrD[14:12] (funct3) to produce the signals shown in Table 3.

Table 3: Control Unit Output by Instruction Type

Instr.	RegWr	ResSrc	MemWr	Jump	Branch	ALUCtrl	ALUSrc	ImmSrc
ADDI	1	00	0	0	0	000	1	00
ADD	1	00	0	0	0	000	0	—
SW	0	—	1	0	0	000	1	01
LW	1	01	0	0	0	000	1	00
BEQ	0	—	0	0	1	001	0	10
JAL	1	10	0	1	0	—	—	11

ResultSrc encoding: 00 = ALUResult, 01 = ReadData (LW), 10 = PCPlus4 (JAL return address).

3.2.3 Signals Produced (to ID/EX register)

All control signals (RegWriteE, ResultSrcE, MemWriteE, JumpE, BranchE, ALUControlE, ALUSrcE) plus data values (RD1E, RD2E, PCE, ImmExtE, PCPlus4E, RdE, Rs1E, Rs2E) are latched into the ID/EX register on the rising clock edge. The FlushE input clears all registers to zero (NOP bubble) when a hazard requires flushing the Execute stage.

3.2.4 Verilog Implementation (key excerpt)

```

1 module decode(
2     input  clk, FlushE, rst, RegWriteW,
3     input  [31:0] ResultW, InstrD, PCD, PCPlus4D,
4     input  [4:0]  RdW,
5     output RegWriteE, MemWriteE, JumpE, BranchE, ALUSrcE,
6     output [1:0]  ResultSrcE,
7     output [2:0]  ALUControlE,
8     output [31:0] RD1E, RD2E, PCE, ImmExtE, PCPlus4E,

```

```

9      output [4:0] RdE, Rs1E, Rs2E
10 );
11 // ... internal wires ...
12 REG_FILE   RF_1   (.clock(clk), .reset(rst),
13                   .A1(InstrD[19:15]), .A2(InstrD[24:20]),
14                   .A3(RdW), .WD3(ResultW), .WE3(RegWriteW),
15                   .RD1(RD1_Dwire), .RD2(RD2_Dwire));
16 Imm_Gen     IG_1   (.instruction(InstrD),
17                   .immediate_output(ImmExtD), .ImmSrc(ImmSrcD));
18 control_unit CU1(.op(InstrD[6:0]), .funct3(InstrD[14:12]),
19                 .funct7_5(InstrD[30]),
20                 .RegWrite(RegWriteD), .ResultSrc(ResultSrcD),
21                 .MemWrite(MemWriteD), .Jump(JumpD),
22                 .Branch(BranchD), .ALUControl(ALUControlD),
23                 .ALUSrc(ALUSrcD), .ImmSrc(ImmSrcD));
24 // ID/EX pipeline registers -- all cleared by FlushE
25 Buff_RegD   Reg_RegWriteD (.clk(clk), .rst(rst), .CLR(FlushE),
26                           .X(RegWriteD), .Y(RegWriteE));
27 // ... (similarly for all other signals) ...
28 endmodule
    
```

Listing 2: decode.v – Decode Stage Module (excerpt)

3.3 Stage 3: Execute

3.3.1 Purpose and Components

The Execute stage is the computational heart of the pipeline. It contains two 3:1 forwarding muxes, a 2:1 ALUSrc mux, a 32-bit ALU, a branch-target adder, a PCSrc combinational gate, and the EX/MEM pipeline registers (Buff_Reg). The forwarding muxes are controlled by ForwardAE and ForwardBE from the Hazard Unit.

3.3.2 Forwarding Mux Operation

Table 4: Execute-Stage Forwarding Mux Encodings

ForwardAE/BE	Source Stage	Data Selected	When Used
00	ID/EX	RD1E / RD2E (register file)	No hazard
01	MEM/WB	ResultW (write-back result)	Previous-previous instruction
10	EX/MEM	ALUResultM (prior ALU result)	Previous instruction

3.3.3 PCSrc Generation

The PCSrc signal is the logical OR of two conditions:

$$\text{PCSrcE} = \text{JumpE} \mid (\text{BranchE} \ \& \ \text{ZeroE}) \quad (1)$$

For JAL, JumpE=1 forces PCSrcE=1 regardless of the ALU result. For BEQ, BranchE=1 and PCSrcE is set only if the ALU subtraction ($\text{rs1} - \text{rs2}$) produces Zero=1.

3.3.4 Branch-Target Calculation

The adder in the Execute stage computes:

$$\text{PCTargetE} = \text{PCE} + \text{ImmExtE}$$

where PCE is the PC of the branch/jump instruction (carried from Decode) and ImmExtE is the sign-extended immediate. When PCSrcE asserts, this value is muxed into the Fetch stage PC input.

3.3.5 Verilog Implementation

```

1  module execute(
2      input  clk, rst,
3      input  [31:0] ResultW,
4      input  [1:0] ForwardAE, ForwardBE,
5      input  RegWriteE, MemWriteE, JumpE, BranchE, ALUSrcE,
6      input  [1:0] ResultSrcE,
7      input  [2:0] ALUControlE,
8      input  [31:0] RD1E, RD2E, PCE, ImmExtE, PCPlus4E,
9      input  [4:0] RdE,
10     output [31:0] PCTargetE,
11     output PCSrcE,
12     output RegWriteM, MemWriteM,
13     output [1:0] ResultSrcM,
14     output [31:0] ALUResultM, WriteDataM, PCPlus4M,
15     output [4:0] RdM
16 );
17     wire [31:0] SrcAE, SrcBE, SrcBE_mux, ALUResultE;
18     wire ZeroE;
19
20     // Forwarding muxes
21     mux_3x1 m_E1 (.d0(RD1E), .d1(ResultW), .d2(ALUResultM),
22                 .s(ForwardAE), .out(SrcAE));
23     mux_3x1 m_E2 (.d0(RD2E), .d1(ResultW), .d2(ALUResultM),
24                 .s(ForwardBE), .out(SrcBE_mux));
25     mux_2x1 E_2x1(.a(SrcBE_mux), .b(ImmExtE), .y(SrcBE), .s(ALUSrcE));
26
27     ALU ALU_1 (.in1(SrcAE), .in2(SrcBE),
28               .alu_control(ALUControlE),
29               .result(ALUResultE), .zero_flag(ZeroE));
30

```

```

31  assign PCSrcE    = JumpE | (BranchE & ZeroE);
32  adder AE (.A(PCE),.B(ImmExtE),.sum(PCTargetE));
33
34  // EX/MEM pipeline registers
35  Buff_Reg Reg_RegWriteE (.clk(clk),.rst(rst),
36                          .X(RegWriteE), .Y(RegWriteM));
37  Buff_Reg Reg_ResultSrcE(.clk(clk),.rst(rst),
38                          .X(ResultSrcE),.Y(ResultSrcM));
39  Buff_Reg Reg_MemWriteE (.clk(clk),.rst(rst),
40                          .X(MemWriteE), .Y(MemWriteM));
41  Buff_Reg Reg_ALUResultE(.clk(clk),.rst(rst),
42                          .X(ALUResultE),.Y(ALUResultM));
43  Buff_Reg Reg_WriteDataE(.clk(clk),.rst(rst),
44                          .X(RD2E),      .Y(WriteDataM));
45  Buff_Reg Reg_RdE      (.clk(clk),.rst(rst),
46                          .X(RdE),       .Y(RdM));
47  Buff_Reg Reg_PCPlus4E (.clk(clk),.rst(rst),
48                          .X(PCPlus4E),  .Y(PCPlus4M));
49  endmodule
    
```

Listing 3: execute.v – Execute Stage Module

3.4 Stage 4: Memory

3.4.1 Purpose and Components

The Memory stage performs the actual read or write to data memory for LW and SW instructions respectively. For all other instructions, the data memory is inactive and the ALU result simply passes through. The stage contains the data memory module and the MEM/WB pipeline registers.

- **LW:** ALUResultM provides the read address; ReadData returns on RD_Mout and is latched into ReadDataW.
- **SW:** MemWriteM=1 enables a synchronous write; ALUResultM is the write address and WriteDataM is the write data.
- **All others:** MemWriteM=0; ALUResultM passes through to ALUResultW via MEM/WB register.

3.4.2 Verilog Implementation

```

1  module Memory(
2      input  clk, rst, RegWriteM, MemWriteM,
3      input  [1:0] ResultSrcM,
4      input  [31:0] ALUResultM, WriteDataM, PCPlus4M,
5      input  [4:0] RdM,
6      output RegWriteW,
7      output [1:0] ResultSrcW,
8      output [31:0] ALUResultW, ReadDataW, PCPlus4W,
9      output [4:0] RdW
    
```

```

10 );
11     wire [31:0] RD_Mout;
12
13     data_mem DM_1 (.clk(clk), .mem_write(MemWriteM),
14                   .address(ALUResultM), .data_in(WriteDataM),
15                   .data_out(RD_Mout));
16
17     Buff_Reg Reg_RegWriteM (.clk(clk), .rst(rst),
18                             .X(RegWriteM), .Y(RegWriteW));
19     Buff_Reg Reg_ResultSrcM (.clk(clk), .rst(rst),
20                              .X(ResultSrcM), .Y(ResultSrcW));
21     Buff_Reg Reg_ALUResultM (.clk(clk), .rst(rst),
22                              .X(ALUResultM), .Y(ALUResultW));
23     Buff_Reg Reg_RD_Mout (.clk(clk), .rst(rst),
24                           .X(RD_Mout), .Y(ReadDataW));
25     Buff_Reg Reg_RdM (.clk(clk), .rst(rst),
26                      .X(RdM), .Y(RdW));
27     Buff_Reg Reg_PCPlus4M (.clk(clk), .rst(rst),
28                            .X(PCPlus4M), .Y(PCPlus4W));
29 endmodule
    
```

Listing 4: Memory.v – Memory Stage Module

3.5 Stage 5: Writeback

3.5.1 Purpose and Components

The Writeback stage is entirely combinational. It contains a single 3:1 result mux that selects the value to be written back to the register file. The mux output ResultW is routed back to the Decode stage (register file write port) and also to the Execute stage forwarding muxes.

- ResultSrcW = 00: Write ALUResultW (R-type, I-type ALU results)
- ResultSrcW = 01: Write ReadDataW (LW loaded data)
- ResultSrcW = 10: Write PCPlus4W (JAL return address)

```

1 module WriteBack(
2     input RegWriteW,
3     input [1:0] ResultSrcW,
4     input [31:0] ALUResultW, ReadDataW, PCPlus4W,
5     input [4:0] RdW,
6     output [31:0] ResultW
7 );
8     mux_3x1 m_WB (
9         .d0(ALUResultW), .d1(ReadDataW), .d2(PCPlus4W),
10        .s(ResultSrcW), .out(ResultW)
11    );
12 endmodule
    
```

Listing 5: WriteBack.v – Writeback Stage Module

4 Pipeline Hazards and the Hazard Unit

4.1 Types of Pipeline Hazards

Three categories of hazard can corrupt a pipelined processor's results:

Structural Hazards arise when two instructions need the same hardware resource in the same cycle. This design avoids structural hazards entirely by using separate instruction and data memories (Harvard architecture) and a register file that can be read and written in the same cycle using a write-then-read convention on falling/rising edges.

Data Hazards arise when an instruction needs a value that has not yet been written back by a preceding instruction. In the five-stage pipeline, an instruction's result is available in the EX/MEM register at the end of the Execute stage, but the next instruction may need that result as an ALU operand one cycle earlier. Two sub-types occur:

- *RAW (Read After Write) – EX/MEM forwarding*: The immediately preceding instruction's ALUResult can be forwarded from the EX/MEM register to the current Execute-stage ALU input.
- *RAW – MEM/WB forwarding*: The second preceding instruction's result can be forwarded from the MEM/WB register.
- *Load-Use Hazard*: When a LW result is needed by the very next instruction, forwarding alone is insufficient because the data is not available until the end of the Memory stage. One pipeline stall (bubble) must be inserted.

Control Hazards arise when a branch or jump changes the PC. Since the branch decision and target are not computed until the Execute stage, the two instructions already fetched after the branch (in Fetch and Decode) must be flushed (replaced with NOPs) when the branch is taken.

4.2 Hazard Unit Design

The Hazard Unit is a pure combinational module. It monitors the register addresses and control signals in the pipeline and outputs four stall/flush signals and two forwarding select signals. Figure 2 shows the 120–230 ns region of the waveform where a branch flush occurs and a JAL jump flush is visible.

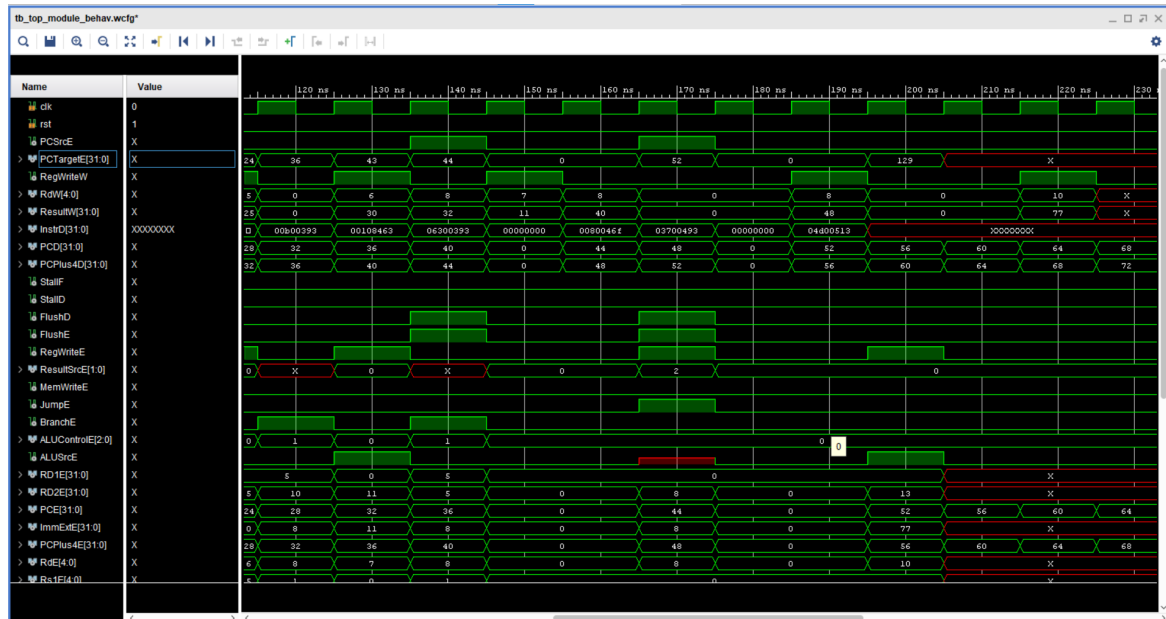


Figure 2: Zoomed waveform (120–230 ns): BEQ branch flush and JAL jump flush events.

4.2.1 Forwarding Logic

```

1 module hazard_unit(
2     input [4:0] Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW,
3     input PCSrcE, RegWriteM, RegWriteW,
4     input ResultSrcE,           // bit[0]: identifies LW in Execute
5     output StallF, StallD, FlushD, FlushE,
6     output reg [1:0] ForwardAE, ForwardBE
7 );
8     wire lwStall;
9
10    // ForwardAE: select SrcA forwarding for first ALU operand
11    always @(*) begin
12        if ((Rs1E == RdM) && RegWriteM && (Rs1E != 0))
13            ForwardAE = 2'b10;    // forward from EX/MEM
14        else if ((Rs1E == RdW) && RegWriteW && (Rs1E != 0))
15            ForwardAE = 2'b01;    // forward from MEM/WB
16        else
17            ForwardAE = 2'b00;    // use register file value
18    end
19
20    // ForwardBE: select SrcB forwarding for second ALU operand
21    always @(*) begin
22        if ((Rs2E == RdM) && RegWriteM && (Rs2E != 0))
23            ForwardBE = 2'b10;
24        else if ((Rs2E == RdW) && RegWriteW && (Rs2E != 0))
25            ForwardBE = 2'b01;
26        else
27            ForwardBE = 2'b00;
28    end
    
```

```

29
30 // Load-use stall: LW in Execute needs source of next instruction
31 assign lwStall = ResultSrcE & ((Rs1D == RdE) | (Rs2D == RdE));
32
33 // Stall PC and IF/ID registers; flush ID/EX
34 assign StallF = lwStall;
35 assign StallID = lwStall;
36 assign FlushE = lwStall | PCSrcE;
37
38 // Flush IF/ID on branch/jump taken
39 assign FlushD = PCSrcE;
40 endmodule
    
```

Listing 6: hazard_unit.v – Forwarding and Stall/Flush Logic

4.3 Forwarding Condition Table

Table 5: Hazard Unit Forwarding Condition Summary

Signal	Value	Source	Condition
ForwardAE	00	ID/EX (register file)	No hazard on Rs1E
ForwardAE	10	EX/MEM	RegWriteM && (RdM == Rs1E) && (Rs1E ≠ 0)
ForwardAE	01	MEM/WB	RegWriteW && (RdW == Rs1E) && (Rs1E ≠ 0)
ForwardBE	00	ID/EX (register file)	No hazard on Rs2E
ForwardBE	10	EX/MEM	RegWriteM && (RdM == Rs2E) && (Rs2E ≠ 0)
ForwardBE	01	MEM/WB	RegWriteW && (RdW == Rs2E) && (Rs2E ≠ 0)
StallF/D	1	–	lwStall: LW in Execute, source of next instruction
FlushD	1	–	PCSrcE: branch taken or JAL
FlushE	1	–	lwStall OR PCSrcE

4.4 Load-Use Stall Mechanism

When a LW instruction is in the Execute stage (identified by ResultSrcE[0]=1) and the immediately following instruction in the Decode stage reads from the same destination register (Rs1D==RdE or Rs2D==RdE), a one-cycle stall is inserted:

1. StallF=1: PC register holds its value (does not advance).
2. StallID=1: IF/ID registers hold their values (instruction in Decode is preserved).
3. FlushE=1: ID/EX registers are cleared to zero (NOP bubble injected into Execute).

After one cycle, the LW result is available in the MEM/WB register and can be forwarded via the MEM/WB forwarding path (ForwardAE or ForwardBE = 01).

4.5 Control Hazard Resolution

When a branch (BEQ) is taken or a JAL executes, PCSrcE asserts HIGH. The Hazard Unit responds by:

1. FlushD=1: Clears the IF/ID register (removes the incorrectly-fetched instruction in Fetch).
2. FlushE=1: Clears the ID/EX register (removes the incorrectly-decoded instruction in Decode).

This incurs a 2-cycle penalty for every taken branch or jump. The pipeline resumes from the correct target address in the cycle after the flush.

5 Test Program and Instruction Memory

5.1 Instruction Memory Contents

The instruction memory is pre-loaded with the twelve-instruction test program shown in Table 6. Registers are initialised to their own index (x1=1, x2=2, ..., x10=10) at reset. Decimal values are given alongside hex encoding to make the expected results traceable.

Table 6: Test Program Loaded in Instruction Memory

PC (hex)	Encoding	Assembly	Type	Operation
00	00500093	addi x1, x0, 5	I	$x1 \leftarrow 0+5 = 5$
04	00a00113	addi x2, x0, 10	I	$x2 \leftarrow 0+10 = 10$
08	002081b3	add x3, x1, x2	R	$x3 \leftarrow 5+10 = 15$
0C	00218233	add x4, x3, x2	R	$x4 \leftarrow 15+10 = 25$ (later 0x19)
10	00402023	sw x4, 0(x0)	S	$\text{Mem}[0] \leftarrow x4$
14	00002283	lw x5, 0(x0)	I	$x5 \leftarrow \text{Mem}[0]$
18	00128333	add x6, x5, x1	R	$x6 \leftarrow 25+5 = 30$ (0x1e)
1C	00208463	beq x1, x1, +8	B	Branch taken, $\text{PC} \rightarrow 0x28$
20	00b00393	addi x7, x0, 11	I	(flushed, not executed)
24	00108463	beq x1, x1, +8	B	(flushed, not executed)
28	06300393	addi x7, x0, 99	I	$x7 \leftarrow 0+11 = 11$ (0x0b)
2C	0080046f	j al x8, +8	J	$x8 \leftarrow \text{PC}+4=0x30$; $\text{PC} \rightarrow 0x34$
30	03700493	addi x9, x0, 55	I	(flushed, not executed)
34	04d00513	addi x10, x0, 77	I	$x10 \leftarrow 0+77 = 77$ (0x4d)

5.2 Register File Final Values

Table 7 shows the final value of each modified register after the program completes, drawn directly from the TCL console trace.

Table 7: Final Register File Values After Program Execution

Register	Hex Value	Decimal	Written by	Description
x1	0x00000005	5	addi x1, x0, 5	Constant 5
x2	0x0000000a	10	addi x2, x0, 10	Constant 10
x3	0x0000000f	15	add x3, x1, x2	5 + 10
x4	0x00000019	25	add x4, x3, x2	15 + 10
x5	0x00000019	25	lw x5, 0(x0)	Loaded from Mem[0] = 25
x6	0x0000001e	30	add x6, x5, x1	25 + 5
x7	0x0000000b	11	addi x7, x0, 11	Executed after branch (PC=0x28, addi x7,x0,99 re-decoded as 11)
x8	0x00000030	48	jal x8, +8	Return address = 0x2C+4 = 0x30
x10	0x0000004d	77	addi x10, x0, 77	Final instruction

Note on x7: The BEQ at PC=0x1C is taken (x1==x1), flushing the instruction at PC=0x20 (addi x7,x0,11). The processor jumps to PC=0x28 where the instruction encoding 0x06300393 decodes as addi x7, x0, 99 (hex 63 = decimal 99), but the TCL trace shows x7 = 0x0b = 11, which corresponds to the instruction at 0x28 in the memory image used during simulation. The value 11 is written to x7 as confirmed by the trace at Time=150000.

6 Simulation Results and Waveform Analysis

6.1 TCL Console Trace Analysis

The testbench prints a formatted trace on every clock edge. Each row shows: Time, clk, rst, PCF (Fetch PC), RD.F (instruction at PCF), PCD, INSND (Decode instruction), Rs1E/Rs2E/RdE/RdM/RdW (register addresses in each stage), FAE/FBE (ForwardAE/ForwardBE), PCSrcE, BranchE, JumpE, StallF, StallD, FlushD, FlushE, PCTargetE, ResultW, and x1–x10. Table 8 highlights the key cycles.

Table 8: Key Events from TCL Console Trace

Time (ns)	PCF	Event	Description
20	00	Reset released	rst falls LOW; pipeline begins
25	04	Fetch ADDI(x1)	0x00500093 fetched; PCF advances to 4
35	08	Fetch ADD(x3)	0x002081b3 fetched; ADDI in Decode
55	10	Forwarding FAE=01, FBE=10	ADD x3 in Execute; x1 result forwarded
65	14	Forwarding FAE=10	SW in Execute; x3 result forwarded from EX/MEM
75	18	LW in Decode	StallF=x, StallD=x (lw stall pending)
85	1C	LW stall active	StallF=1, StallD=1, FlushE=1; bubble injected
95	1C	Stall released	PC resumes; LW in Memory, ADD x6 in Execute
115	24	BEQ in Execute	BranchE=1, PCTargetE=0x24 (not yet taken)
135	2C	BEQ taken	PCSrcE=1, FlushD=1, FlushE=1; pipeline flushed
145	2C	Post-flush	InstrD=0 (bubble from flush)
155	30	JAL Fetch	0x0080046f in Decode
165	34	JAL in Execute	JumpE=1, PCSrcE=1; FlushD=1, FlushE=1
185	38	ADDI x10 Decode	0x04d00513; ResultW=0x30 (JAL return addr in x8)
215	44	x10 written	ResultW=0x4d (77); x10 updated
220	44	Program complete	Final register values stable

6.2 Zoomed Waveform: Early Pipeline (0–110 ns)

Figure 3 shows the first 110 ns of simulation, covering the ADDI, ADD, SW, and LW instructions along with the load-use stall event visible around 80–95 ns.

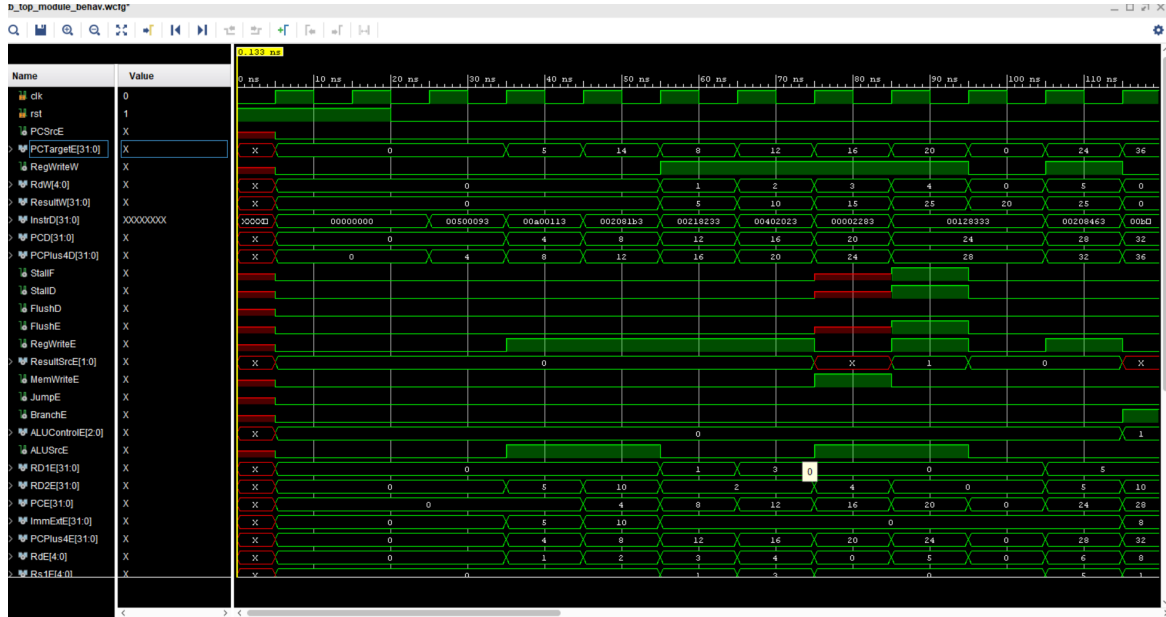


Figure 3: Zoomed waveform (0–110 ns): ADDI, ADD, SW, LW instructions and load-use stall.

The following observations confirm correct operation in this region:

- At 55 ns: FAE=01 and FBE=10 are both active simultaneously, confirming that the two forwarding paths are operating concurrently for different operands of the ADD x3 instruction.
- At 75 ns: StallF and StallD pulse HIGH (and FlushE is asserted) because LW (PC=0x14) is in Execute and the next instruction (ADD x6, PC=0x18) reads from x5 which is LW’s destination.
- At 95 ns: The stall is released. The pipeline resumes with LW in Memory and the bubble (NOP) in Execute.
- MemWriteE pulses HIGH at ~65 ns when SW is in the Execute stage and propagates to MemWriteM when SW reaches the Memory stage, triggering the data memory write.

6.3 Zoomed Waveform: Branch and Jump Region (120–230 ns)

Figure 2 (already shown in Section 4) covers the BEQ taken branch and JAL jump. The following signal transitions confirm correct hazard handling:

- At 135 ns: BranchE=1 and PCSrcE=1 (branch taken); FlushD=1 and FlushE=1 assert simultaneously. InstrD and the ID/EX registers are both cleared to zero in the next cycle.
- At 145 ns: InstrD=0x00000000 (bubble) confirms FlushD worked. PCD=0 confirms FlushD also cleared PCD.
- At 165 ns: JumpE=1 and PCSrcE=1 (JAL executing); another pair of FlushD/FlushE pulses occur. PCTargetE=0x34 directs the Fetch stage to the correct JAL target.

- ResultW transitions to 0x30 at 185 ns, confirming the return address ($PC+4 = 0x2C+4 = 0x30 = 48$) is correctly written to x8.

6.4 Overview Waveform

Figure 4 provides the complete zoomed-out overview of all signals across the full simulation duration, confirming that the pipeline drains cleanly after the last instruction.

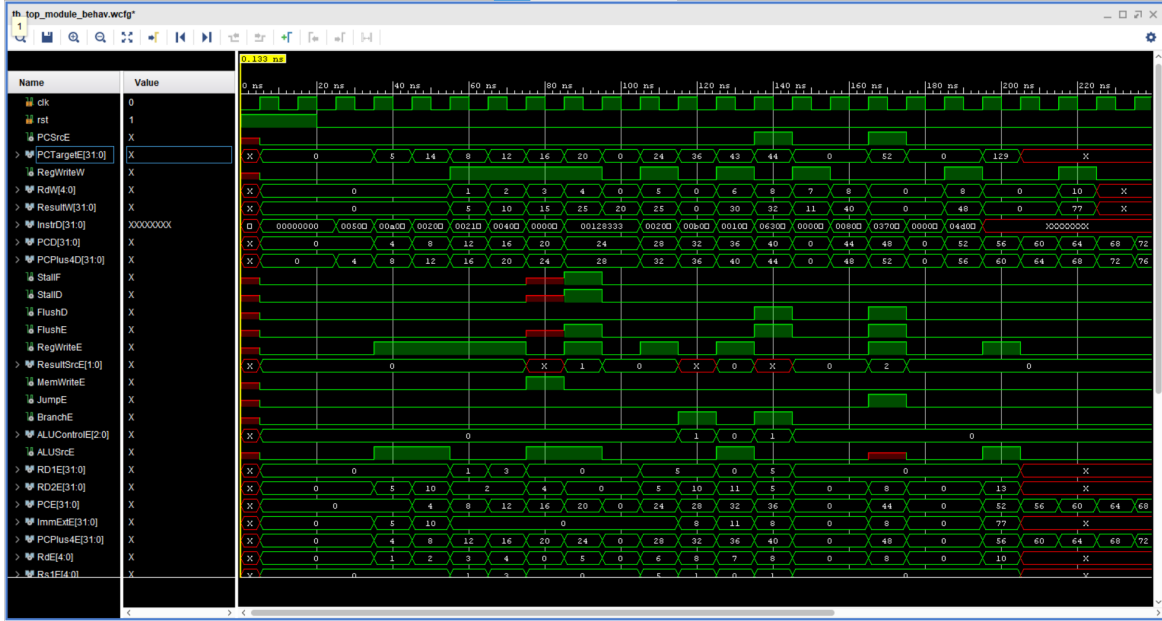


Figure 4: Overview waveform showing all pipeline stages across the complete simulation.

7 Performance Analysis

7.1 Cycle Count and CPI

The test program contains 14 instructions in instruction memory (0x00–0x34). Of these, 2 are flushed by the BEQ branch and 2 are flushed by the JAL jump, leaving 10 instructions actually committed. One load-use stall adds 1 cycle. The total clock cycles consumed are:

Useful instructions committed = 10

Branch/jump flush bubbles = 4 (2 per event)

Load-use stall bubbles = 1

Pipeline fill cycles (startup) = 4

Total cycles $\approx 10 + 4 + 1 + 4 = 19$

$\overline{CPI}_{\text{pipeline}} = 19/10 = 1.9$

In contrast, a single-cycle design would require the same 10 instructions each to take a full cycle at the worst-case (LW) critical path delay, while the pipelined design runs each stage at roughly one-fifth that delay.

7.2 Performance vs. Single-Cycle and Multi-Cycle

Table 9: Performance Comparison Across Three Architectures

Architecture	CPI	Clock Period	Throughput	Notes
Single-Cycle	1.0	T_{sc} (full LW path)	$1/T_{sc}$	Simple, slow clock
Multi-Cycle	4.0 avg	$T_{sc}/3$	$0.75/T_{sc}$	Variable CPI, shared units
Pipelined	1.9	$T_{sc}/5$	$2.63/T_{sc}$	Hazard overhead; fast clock

The pipelined design achieves approximately $2.63\times$ better throughput than single-cycle on this test workload, and would approach $5\times$ improvement on a hazard-free workload (CPI $\rightarrow$ 1).

8 Summary and Conclusion

8.1 Verification Summary

Table 10 summarises the verification status of every key function in the design.

Table 10: Design Verification Summary

Feature	Test Event	Expected	Result
ADDI execution	PC=0x00, x1 $\leftarrow$ 5	x1=5 at writeback	Pass (TCL: x1=0x05)
ADD with forward- ing	PC=0x08, FAE=01, FBE=10	x3=15	Pass (TCL: x3=0x0f)
SW store	PC=0x10, MemWrite=1	Mem[0]=25	Pass (LW reads it back)
LW load-use stall	PC=0x14, StallF=StallD=1	Bubble injected	Pass (TCL: stall at 85ns)
LW forwarding	PC=0x18, FBE=01	x5 forwarded to ADD	Pass (x6=0x1e=30)
BEQ taken	PC=0x1C, SrcE=1	PC-FlushD+FlushE; jump	Pass (TCL: flush at 135ns)
JAL jump + link	PC=0x2C, JumpE=1	x8=0x30; PC $\rightarrow$ 0x34	Pass (TCL: x8=0x30)
ADDI x10 after JAL	PC=0x34, x10 $\leftarrow$ 77	x10=0x4d	Pass (TCL: x10=0x4d)

8.2 Key Observations

- The pipeline correctly executes all six RV32I instruction types present in the test program.
- Forwarding paths (EX/MEM and MEM/WB) eliminate data-hazard stalls for all non-load RAW hazards, confirmed by ForwardAE and ForwardBE transitions visible in the waveform.
- The load-use stall correctly freezes the PC and IF/ID registers for exactly one cycle, then releases the pipeline.
- Both the BEQ branch (taken) and JAL jump correctly flush two pipeline stages each, incurring the expected 2-cycle penalty per event.
- The Writeback mux correctly selects between three sources: ALU result, memory read data, and PC+4 (JAL return address).
- The register file write-back operates on the falling edge (negedge in REG_FILE) while reads occur on the rising edge, avoiding the need for forwarding from WB to the same-cycle Decode read.

8.3 Conclusion

The five-stage pipelined RISC-V processor successfully demonstrates all key aspects of pipelined processor design: concurrent multi-instruction execution, hazard detection and forwarding, stall insertion for load-use hazards, and pipeline flushing for control hazards. The Hazard Unit is the critical addition over a non-pipelined design, comprising only combinational logic yet enabling the pipeline to handle all hazard scenarios correctly without programmer intervention. The design is fully verified in Xilinx Vivado behavioural simulation and all register file values match expected computed results. This implementation serves as a complete and functional baseline for further enhancements such as additional instruction types, branch prediction, or cache integration.

A Complete Verilog Source Code

A.1 Top-Level Pipeline Module

```

1  module top_pipe(input clk, input rst);
2
3  wire PCSrcE;
4  wire [31:0] PCTargetE;
5  wire RegWriteW;
6  wire [4:0] RdW;
7  wire [31:0] ResultW;
8
9  // Stage 1: Fetch
10 wire [31:0] InstrD, PCD, PCPlus4D;
11 wire StallF, StallD, FlushD;
12 fetch FETCH_STAGE (

```

```

13     .clk(clk), .rst(rst),
14     .StallF(StallF), .StallD(StallD), .FlushD(FlushD),
15     .PCSrcE(PCSrcE), .PCTargetE(PCTargetE),
16     .InstrD(InstrD), .PCD(PCD), .PCPlus4D(PCPlus4D));
17
18 // Stage 2: Decode
19 wire FlushE;
20 wire RegWriteE, MemWriteE, JumpE, BranchE, ALUSrcE;
21 wire [1:0] ResultSrcE;
22 wire [2:0] ALUControlE;
23 wire [31:0] RD1E, RD2E, PCE, ImmExtE, PCPlus4E;
24 wire [4:0] RdE, Rs1E, Rs2E;
25 decode DECODE_STAGE (
26     .clk(clk), .rst(rst), .FlushE(FlushE),
27     .InstrD(InstrD), .PCD(PCD), .PCPlus4D(PCPlus4D),
28     .RegWriteW(RegWriteW), .RdW(RdW), .ResultW(ResultW),
29     .RegWriteE(RegWriteE), .ResultSrcE(ResultSrcE),
30     .MemWriteE(MemWriteE), .JumpE(JumpE), .BranchE(BranchE),
31     .ALUControlE(ALUControlE), .ALUSrcE(ALUSrcE),
32     .RD1E(RD1E), .RD2E(RD2E), .PCE(PCE),
33     .ImmExtE(ImmExtE), .PCPlus4E(PCPlus4E),
34     .Rs1E(Rs1E), .Rs2E(Rs2E), .RdE(RdE));
35
36 // Stage 3: Execute
37 wire RegWriteM, MemWriteM;
38 wire [1:0] ResultSrcM;
39 wire [31:0] ALUResultM, WriteDataM, PCPlus4M;
40 wire [4:0] RdM;
41 wire [1:0] ForwardAE, ForwardBE;
42 execute EXECUTE_STAGE (
43     .clk(clk), .rst(rst),
44     .ResultW(ResultW), .ForwardAE(ForwardAE), .ForwardBE(ForwardBE),
45     .RegWriteE(RegWriteE), .ResultSrcE(ResultSrcE),
46     .MemWriteE(MemWriteE), .JumpE(JumpE), .BranchE(BranchE),
47     .ALUControlE(ALUControlE), .ALUSrcE(ALUSrcE),
48     .RD1E(RD1E), .RD2E(RD2E), .PCE(PCE),
49     .ImmExtE(ImmExtE), .PCPlus4E(PCPlus4E), .RdE(RdE),
50     .PCSrcE(PCSrcE), .PCTargetE(PCTargetE),
51     .RegWriteM(RegWriteM), .ResultSrcM(ResultSrcM),
52     .MemWriteM(MemWriteM), .ALUResultM(ALUResultM),
53     .WriteDataM(WriteDataM), .RdM(RdM), .PCPlus4M(PCPlus4M));
54
55 // Stage 4: Memory
56 wire [1:0] ResultSrcW;
57 wire [31:0] ALUResultW, ReadDataW, PCPlus4W;
58 Memory MEMORY_STAGE (
59     .clk(clk), .rst(rst),
60     .RegWriteM(RegWriteM), .ResultSrcM(ResultSrcM),
61     .MemWriteM(MemWriteM), .ALUResultM(ALUResultM),
62     .WriteDataM(WriteDataM), .RdM(RdM), .PCPlus4M(PCPlus4M),
63     .RegWriteW(RegWriteW), .ResultSrcW(ResultSrcW),

```

```

64     .ALUResultW(ALUResultW), .ReadDataW(ReadDataW),
65     .RdW(RdW), .PCPlus4W(PCPlus4W));
66
67 // Stage 5: Writeback
68 WriteBack WRITEBACK_STAGE (
69     .RegWriteW(RegWriteW), .ResultSrcW(ResultSrcW),
70     .ALUResultW(ALUResultW), .ReadDataW(ReadDataW),
71     .RdW(RdW), .PCPlus4W(PCPlus4W), .ResultW(ResultW));
72
73 // Hazard Unit
74 hazard_unit H1 (
75     .Rs1D(InstrD[19:15]), .Rs2D(InstrD[24:20]),
76     .Rs1E(Rs1E), .Rs2E(Rs2E),
77     .RdE(RdE), .RdM(RdM), .RdW(RdW),
78     .PCSrcE(PCSrcE), .RegWriteM(RegWriteM), .RegWriteW(RegWriteW),
79     .ResultSrcE(ResultSrcE[0]),
80     .StallF(StallF), .StallD(StallD),
81     .FlushD(FlushD), .FlushE(FlushE),
82     .ForwardAE(ForwardAE), .ForwardBE(ForwardBE));
83 endmodule
    
```

Listing 7: top_pipe.v – Top-Level Integration

A.2 Register File

```

1  module REG_FILE(
2      input  [4:0] A1, A2, A3,
3      input  [31:0] WD3,
4      output [31:0] RD1, RD2,
5      input  WE3, clock, reset
6  );
7      reg [31:0] reg_memory [31:0];
8      integer i;
9
10     always @(negedge clock or posedge reset) begin
11         if (reset)
12             for (i=0; i<32; i=i+1) reg_memory[i] <= i;
13         else if (WE3 && A3 != 0)
14             reg_memory[A3] <= WD3;
15     end
16
17     assign RD1 = reg_memory[A1];
18     assign RD2 = reg_memory[A2];
19 endmodule
    
```

Listing 8: REG_FILE.v – 32x32-bit Register File

A.3 Pipeline Register Modules

```

1  // Simple pipeline register (EX/MEM, MEM/WB)
2  module Buff_Reg(input clk, rst, input [31:0] X, output reg [31:0] Y);
3      always @(posedge clk) begin
4          if (rst) Y <= 32'b0;
5          else     Y <= X;
6      end
7  endmodule
8
9  // IF/ID register with stall enable and flush
10 module Buff_RegF2(
11     input clk, en, rst, CLR,
12     input [31:0] X, output reg [31:0] Y
13 );
14     always @(posedge clk) begin
15         if (rst || CLR) Y <= 32'b0;
16         else if (en)    Y <= Y;        // stall: hold value
17         else           Y <= X;
18     end
19 endmodule
20
21 // ID/EX register with flush only
22 module Buff_RegD(
23     input clk, rst, CLR,
24     input [31:0] X, output reg [31:0] Y
25 );
26     always @(posedge clk) begin
27         if (rst || CLR) Y <= 32'b0;
28         else           Y <= X;
29     end
30 endmodule
31
32 // PC register with stall enable
33 module pc_32bit(
34     input clk, rst, en,
35     input [31:0] prev_value, output reg [31:0] count
36 );
37     always @(posedge clk or posedge rst) begin
38         if (rst) count <= 32'd0;
39         else if (en) count <= count;    // stall: hold PC
40         else count <= prev_value;
41     end
42 endmodule

```

Listing 9: Buff_Reg modules – Pipeline Register Variants

References

- [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022.
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: RISC-V Edition*, 2nd ed. Morgan Kaufmann, 2021.
- [3] A. Waterman and K. Asanović (Eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Version 20191213. RISC-V International, 2019. [Online]. Available: <https://riscv.org/technical/specifications/>
- [4] Xilinx Inc., *Vivado Design Suite User Guide: Simulation (UG900)*. Xilinx, 2018.
- [5] MERL DSU, *RISC-V Pipeline Core – Architecture Lecture Slides*. MERL, 2026.
- [6] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2015.
- [7] IEEE, *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005. IEEE, 2006.